Push 2 / Nuendo Bridge

Plugin Mapper Guide — Version 1.0

1. Overview

The Plugin Mapper lets you create custom parameter mappings for your plugins. When you focus a mapped plugin in Inserts mode, the Push 2 encoders automatically control the parameters you have assigned, organized in pages of 8.

- **Scanner**: discovers installed VST3 plugins and extracts their parameter lists using Spotify's pedalboard library (GPL-3.0).
- **DirectAccess capture**: for plugins the scanner cannot read (iLok/PACE-protected, Waves shells, Nuendo's stock plug-ins), press **Shift+Browse** on the Push to capture the parameters live from the running plugin inside Nuendo — works for insert effects and instruments alike.
- **Web interface**: a drag-and-drop UI served at http://localhost:8100 for creating and editing mappings. The **Edit Map** button on the Push (Inserts parameter page, lower row 4) opens it directly on the selected plugin.

The server is integrated into the bridge and starts automatically. It is entirely optional — the bridge works fully without it.

2. Installation & Requirements

Standalone app (.app / .exe): nothing to install. All Mapper dependencies (fastapi, uvicorn, pedalboard) are bundled inside the application — the Mapper and its scanner work out of the box.

Only when **running the bridge from source** do you need to install the packages yourself:

```
pip install fastapi uvicorn pedalboard
```

Without pedalboard (web interface only, no scanning):

```
pip install fastapi uvicorn
```

Startup messages:

- **running at http://localhost:8100, scanner ready** — everything available (always the case with the standalone app)
- **running, scanner unavailable** — pedalboard missing (source installs only)
- **not available** — fastapi/uvicorn missing (bridge works normally)

3. File Locations

File	Location	Description
Plugin cache	~/.push2bridge/plugin_cache.json	Scanned plugin parameters (auto-generated)
Mappings	~/.push2bridge/mappings/*.json	One JSON file per mapped plugin
Settings	~/.push2bridge/mapper_settings.json	Scanner settings (extra folders)
Mapper source	src/mapper/	Scanner, server, and web interface files

*All user data is stored in **~/.push2bridge/**. Mappings are plain JSON files that can be shared, backed up, or edited manually — and exported/imported in bulk from the web interface (Export / Import buttons).*

4. Scanning Plugins

The scanner discovers VST3 plugins in the standard directories **and** in the Nuendo/Cubase application bundles (stock plug-ins), version-independently:

- macOS: /Library/Audio/Plug-Ins/VST3, ~/Library/Audio/Plug-Ins/VST3, /Applications/Nuendo*.app/Contents/VST3, /Applications/Cubase*.app/Contents/VST3, Steinberg component folders
- Windows: C:/Program Files/Common Files/VST3, C:/Program Files/Steinberg (recursive), C:/Program Files/Common Files/Steinberg/VST3
- Additional folders can be added with the **Folders** button in the web interface (saved in mapper_settings.json).

Click **Scan** in the web interface. Each plugin is loaded in an isolated subprocess with a 120-second timeout. A **progress bar** shows the current plugin, with **Cancel** (stop the whole scan) and **Skip** (skip the plugin currently being scanned) buttons.

- **Multi-plugin VST3 shells** (one .vst3 file containing several plugins) are expanded into their individual plugins.
- Plugins that fail to scan are listed as **ignored** with a reason — including explicit hints for **Intel-only** binaries (on Apple Silicon) and **PACE/iLok-protected** plugins. Each ignored entry has a ■ **rescan** (300s timeout) and a ✕ **remove** button.
- WaveShell containers are skipped (they cannot be scanned offline) — capture Waves plugins with Shift+Browse instead (section 5).

```
Command line: cd src/mapper && python scanner.py
```

Options: --force (rescan all), --retry (retry errors), --stats (cache statistics).

5. DirectAccess Capture (Shift+Browse)

Some plugins cannot be scanned from disk: iLok/PACE-protected plug-ins, Waves shells, and Nuendo's host-locked stock plug-ins. For those, capture the parameters **live from Nuendo**:

- 1. Load the plugin in Nuendo (insert slot, or the track's instrument).
- 2. In **Inserts mode**, select the insert and press **Shift+Browse** — or, on an instrument track in any other mode, Shift+Browse captures the **instrument's** parameters.
- 3. The Push shows “Capturing…” then the parameter count (large instruments with thousands of parameters are supported).
- 4. The plugin appears in the Mapper's list with type **DirectAccess**, ready to map like any scanned plugin.

  *Tip: the **Edit Map** button (Inserts parameter page, lower row 4) opens the Mapper in your browser directly on the selected plugin.*

6. Creating Mappings

Open http://localhost:8100 (or the Plugin Mapper entry in the macOS menu bar / Windows tray).

Left Panel — Plugin List

- Lists all scanned and captured plugins with a search box and filters: **manufacturer**, **kind** (Effect/Instrument), **state** (Mapped/Unmapped), and **type** (Scanned/DirectAccess).

Center Panel — Parameters

- Shows all parameters of the selected plugin as draggable cards, with a **parameter search field** to find a parameter by name in large lists.

Right Panel — Mapping Pages

- Each page has 8 slots for the 8 Push 2 encoders.
- Drag parameters from the center panel into encoder slots — and **drag between slots** to swap two assignments.
- Click Add Page for additional pages (navigate with Left/Right on Push).
- Edit short labels for each slot (displayed on Push 2 screen).
- Click Save Mapping — saved to ~/.push2bridge/mappings/{plugin_name}.json.
- Click Delete Mapping to remove an existing mapping.

7. How Mappings Work

When you enter the parameter view for an insert plugin:

- 1. The bridge checks ~/.push2bridge/mappings/ for a matching JSON file.
- 2. If found, it asks Nuendo (via DirectAccess) to enumerate all plugin parameters.
- 3. It matches scanned parameter names to DA names using fuzzy word matching (e.g. 'band_1_frequency' matches 'Band 1 Frequency').
- 4. Configures the 8 encoders to control the matched parameters directly.

- 5. Displays parameter names and live values on the Push 2 screen.

If no mapping exists, the bridge uses Nuendo's default parameter bank.

> *Mappings are loaded at startup. After creating or modifying a mapping, press ■ **Reload** on the Push (Setup page, upper row 7) — or restart the bridge.*

8. Troubleshooting

- **Plugin not in list**: click Scan, or add its folder via the Folders button. If it still fails (PACE/iLok, Waves, Nuendo stock), capture it with Shift+Browse (section 5).
- **“Intel-only” error on Apple Silicon**: the plugin has no arm64 binary — the offline scanner cannot load it. Use Shift+Browse capture instead.
- **Parameters don't match**: edit the mapping JSON manually if fuzzy matching fails.
- **Encoders don't respond**: ensure DA insert cache is built. Re-enter the insert to re-trigger.
- **Web interface not loading** (source installs only): check that fastapi and uvicorn are installed.

9. Dependencies

Bundled inside the standalone .app/.exe; only source installs need to install them:

Package	Version	License	Purpose
pedalboard	>=0.9.0	GPL-3.0	VST3 plugin loading and parameter extraction
fastapi	>=0.104.0	MIT	REST API server
uvicorn	>=0.24.0	BSD-3-Clause	ASGI server for FastAPI

> *All dependencies are GPL-3.0 compatible.*